

developers' notes for nucleardatapy toolkit

Jérôme Margueron¹, Christian Drischler², Mariana Dutra³,
Stefano Gandolfi⁴, Alexandros Gezerlis⁵, Guilherme Grams^{6,12},
Sébastien Guillot^{7,8}, Rohit Kumar⁹, Sudhanva Lalit⁹, Odilon
Lourenço³, Rahul Somasundaram^{10,4}, Ingo Tews⁴, Isaac Vidaña¹¹

¹International Research Laboratory on Nuclear Physics and Astrophysics, Michigan State University and CNRS, East Lansing, MI 48824, USA

²Institute of Nuclear and Particle Physics (INPP), Ohio University, Athens, OH 45701, USA

³Departamento de Física e Laboratório de Computação Científica Avançada e Modelamento (Lab-CCAM), Instituto Tecnológico de Aeronáutica, DCTA, 12228-900, São José dos Campos, SP, Brazil

⁴Theoretical Division, Los Alamos National Laboratory, Los Alamos, New Mexico 87545, USA

⁵Department of Physics, University of Guelph, Guelph, Ontario N1G 2W1, Canada

⁶Institut d'Astronomie et d'Astrophysique, CP-226, Université Libre de Bruxelles, 1050 Brussels, Belgium

⁷IRAP, CNRS, 9 avenue du Colonel Roche, BP 44346, F-31028 Toulouse Cedex 4, France

⁸Université de Toulouse, CNES, UPS-OMP, F-31028 Toulouse, France.

⁹Facility for Rare Isotope Beams, Michigan State University, East Lansing, MI 48824, USA

¹⁰Department of Physics, Syracuse University, Syracuse, NY 13244, USA

¹¹Istituto Nazionale di Fisica Nucleare, Sezione di Catania, Dipartimento di Fisica e Astronomia "Ettore Majorana", Università di Catania, Via Santa Sofia 64, I-95123 Catania, Italy

¹²Institut für Physik und Astronomie, Universität Potsdam, Haus 28, Karl-Liebknecht-Str. 24/25, 14476, Potsdam, Germany
the date of receipt and acceptance should be inserted later

Abstract These notes provide additional information for developers that are not written in the paper for **nucleardatapy**.

These notes provide complete information for developers, working in their respective branch and merging with the **developer** branch when it can be done. The merging cannot be done by the same person developing. This is a rule that shall be followed by all developers.

The merging with the **developer** branch implies a battery of checks and tests.

We then merge sometimes to the main branch, usually implying an increase of the current version number.

1 Installation notes for the toolkit.

The **nucleardatapy** toolkit is stored in the following GitHub repository:

It is, however, not necessary to install the GitHub repository to install the toolkit.

1.1 Using pip

You can work with the version which is publicly accessible and install **nucleardatapy** in the following way:

```
pip install nucleardatapy
```

However, this version is frozen and cannot be changed easily. There is moreover a versioning procedure, which make us sure that we always release a working version.

If you perform modification of the toolkit, you may prefer to install this new version on your own folder. To do so, you shall first move to your local clone of the **nucleardatapy** GitHub folder. Then simply launch the following command from a terminal:

```
pip install -e .
```

from within the main folder and you should be able to install the package. Once installed, you can access this package from any working directory.

If you want to install from outside the folder, use

```
pip install --target=/custom/path/  
nucleardatapy nucleardatapy
```

to install the package.

The use of the **nucleardatapy** toolkit implies to install a set of other libraries. These libraries are listed in the file **requirements.txt** and can all be installed in the following way:

```
pip install -r requirements.txt
```

The libraries required to compile the documentation are listed in the file **docs/requirements.txt** and can be installed in the following way:

```
pip install -r docs/requirements.txt
```

1.2 Work in your branch

The branch `develop` contains the latest package that developers have completed (according to their choice).

Create your branch from `develop`:

```
git checkout develop
git checkout -b [mybranchname]
```

Go to your branch:

```
git checkout [mybranchname]
```

Merge `develop` branch into your branch to get (in your branch) the latest version of the toolkit from the `develop` branch before making any changes in your branch:

```
git merge develop
```

Make modifications in your branch and push it to remote:

```
git add -A
git commit -m "comment here about
your work"
git push
```

Create a pull request to `develop`: To create a pull request, navigate to the main page of the repository on GitHub, select the branch with your changes, and click "Compare & pull request." Make sure that the base branch in pull request is `develop` branch

Once approved, the changes in your branch will be merged into "develop" branch.

1.3 Using `install.sh`

The instructions for the installation of the code are detailed in the `README.md` file located in the GitHub repository []. At the end of the installation process, it is necessary to define an environment variable `NUCLEARDATAPY_TK` that contains the absolute path where the toolkit is located. In addition, adding the toolkit path to the python environment variable `PYTHONPATH` allows one to see the toolkit from everywhere in the computer. To do so, just add to your configuration file:

```
export PYTHONPATH = ${
    NUCLEARDATAPY_TK}
```

as suggested at the end of the installation process. If the variable `PYTHONPATH` is already defined and points towards other Python codes, just replace the previous instruction by the following ones:

```
export PYTHONPATH = ${
    NUCLEARDATAPY_TK}:${PYTHONPATH}
```

If the environment variable `PYTHONPATH` is not defined then one should add the following instructions to the header of all Python codes:

```
import os; import sys
nucleardatapy_tk =
    os.getenv('NUCLEARDATAPY_TK')
sys.path.insert(0, nucleardatapy_tk)
```

1.4 Common features

In all cases, one must import the toolkit in your python script in the usual way:

```
import nucleardatapy as nuda
```

From now the `nucleardatapy` toolkit will be called `nuda`, in short.

The list of all functions and global variables available in the toolkit can be printed with the following instructions:

```
print(dir(nuda))
```

You can also have a detailed view of the routines with:

```
print(help(nuda))
```

or

```
print(nuda.__dict__)
```

as well as

```
print(nuda.__dict__.keys())
```

In the following, we detail these functions and discuss the results. You can instantiate the variable `mass` in the following way:

```
mass = nuda.astro.setupMasses()
```

and then, to get all the properties available for the variable `mass`, write:

```
mass.__dict__
```

The toolkit is divided into different modules: `astro`, `corr`, `eos`, `fig`, `hnuc`, `matter`, `nuc`. In addition, there are a few common files where global definitions are fixed: `cst.py` for fixing the constants, `env.py` for fixing environment flags (such as `verb`), `param.py` for parameters providing, for instance, the absolute paths to data files.

In addition to tutorials, there are scripts and plots located in `nucleardatapy.samples/`, which provide examples of the use of the toolkit. They can be copied anywhere else on the user `home` directory and still work, as long as the variable `PYTHONPATH` pointing to the toolkit repository is defined, as explained before.

1.5 When nothing work

If you face issues installing or using the toolkit, it could be that something weird and hard to catch out arrived. The best and fastest action could be to re-start the installation from zero.

To see the version of `nucleardatapy` that you have installed:

```
pip show nucleardatapy
```

First, if you still have a folder `nucleardatapy` in `mylib`, then delete it.

You can uninstall the version you currently have:

```
pip uninstall nucleardatapy
```

Install the current version (developer mode):

```
pip install --force-reinstall -e .
```

or (user mode)

```
pip install --force-reinstall
nucleardatapy
```

Verify the installation again:

```
pip show nucleardatapy
```

You can also purge the previous cache before you reinstall the package again:

```
pip cache purge
```

2 Documentation

The documentation is generated automatically from the docstrings inserted in the functions of the toolkit. All the files necessary for the documentation are stored in the folder `docs/`. The file `docs/index.rst` contains the structure of the documentation, and refer to files which are stored in `docs/source/`. Be careful, only the functions listed in the files stored in `docs/source/` will be shown on the documentation.

To create a new documentation in html format, launch the following command from the terminal, after moving to `docs/`:

```
make html
```

The new documentation can be found in `docs/_build/html/index.html`.

To make a documentation as a pdf file, launch:

```
make pdflatex
```

The new documentation can be found in `docs/_build/pdf/`.

3 Tutorials

Tutorials are stored in the folder `book/notebooks/` where they are organized in chapters. The chapters are associated to each of the modules of the toolkit. For instance, chapter 2 is dedicated to the module `matter` while chapter 5 to the module `nuc`. The tutorials are written in jupyter notebook format (*.ipynb).

Each tutorial is self-contained and can be read independently from the others. An introduction chapter recalls the main commands, common to all modules.

These tutorials are assembled together automatically with the following command:

```
to be completed
```

and can be accessed in the following way to be completed.

4 Install nuda toolkit on an iPad

4.1 Using a-Shell

The free app `a-Shell` emulates a terminal for the iOS of your iPad. It can be downloaded directly from the App Store. Once installed, you can launch it and write in the prompt: `help`. A list of commands and some explanation will be provided on the terminal screen.

Use `pickFolder` to copy the uncompressed folder inside the terminal emulator, probably in `nucleardatapy`. Then in `a-Shell` write:

```
cp -R nucleardatapy /Documents/
```

You can clone the git repository with the following command:

```
lg2 clone https://github.com/
jeromemargueron/nucleardatapy.git
```

Create in Documents a file `.profile` with the following line:

```
export NUCLEARDATAPY\_TK=
nucleardatapy/version1.0/
nucleardatapy
```

Note: `a-Shell` has a Vim editor. Press `i` to write into the file, and if you do not have an `Esc` key on your keyboard, then use `cmd+.`, and write `:wq` to write and quit. Launch:

```
source .profile
```

Check that the environment variable is active:

```
shell $NUCLEARDATAPY\_TK
```

Then go to the sample folder

```
cd nucleardatapy/version1.0/  
nucleardatapy\_samples/
```

You can launch any samples. Maybe you will have to install some additional libraries:

```
python -m pip install numpy  
python -m pip install scipy
```

4.2 Using Carnets-Plus

We have tried the iPad app called **Carnets-Plus** since it is a free app providing a Jupyter Notebook running locally on an iPad. The root folder is accessible from the **Files** app and is located in the local iPad folder inside the folder named **carnets**.

Using **Files**, go to the **carnets** folder and copy the **nucleardatapy.gz** file. Unzip this file and change its name to **nucleardata**.

Then in the Jupyter notebook, write:

```
import os; import sys;  
sys.path.insert(0, os.path.abspath('nucleardata/version1.0'))  
import nucleardatapy as nuda
```

You can now start running the toolkit, and follow the instructions hereafter.